智能交互终端技术报告

基于 STM32F407 的多模态智能交互终端

2026 年 8 月

项目名称	智能交互终端
主控芯片	STM32F407VET6 (ARM Cortex-M4, 168MHz)
语音识别	天问 ASR PRO (CI-03T 离线语音芯片)
WiFi 通信	ESP8266 (ESP-AT 固件, 802.11 b/g/n)
显示屏	2.8 寸 TFT-LCD (240×320, ILI9341, 16 位并口)
触摸屏	GT9147 电容触摸 (I2C)
操作系统	FreeRTOS (CMSIS-RTOS v2)
图形库	LVGL v8.3
云端 AI	阿里云 DashScope (Qwen 3.8-Max)
开发环境	STM32CubeMX + Keil MDK-ARM

摘要

本项目设计并实现了一款智能交互终端。系统以 STM32F407 为主控制器，集成 LVGL 图形界面、虚拟表情动画、ASR PRO 离线语音命令控制、模拟时钟、日历，以及通过 ESP8266 连接阿里云 DashScope 大模型实现的云端 AI 对话功能。系统采用 FreeRTOS 多任务架构，通过消息队列实现模块间解耦通信，支持触摸屏拼音输入与离线语音命令两种交互方式，实现了具身化的智能对话体验。

目录

1 系统总体架构 **4**
1.1 硬件架构 4
1.2 软件架构 4

2 用户界面设计 **5**
2.1 页面结构 5
2.2 界面展示 6
2.3 拼音输入法 7
2.4 机器人面部表情动画 7

3 语音控制模块 **8**
3.1 ASR PRO 离线语音识别 8
3.1.1 硬件接线 8
3.1.2 通信协议 9
3.1.3 STM32 端串口接收与解析 9

4 云端 AI 对话模块 **9**
4.1 通信链路 9
4.2 AT 指令序列 10
4.3 HTTPS 通信的关键坑点 10
4.4 API 调用 10

5 关键代码模块 **11**
5.1 项目文件结构 11
5.2 LVGL 显示接口 12
5.3 ASR PRO 串口帧解析 12
5.4 任务间消息队列通信 13

6 调试与问题解决 **13**
6.1 ASR PRO 串口通信问题 13
6.2 ESP8266 HTTPS 连接问题 14
6.3 机器人表情嘴巴不闭合 14

7 技术指标 **15**

8 总结与展望 **15**
8.1 项目成果 15

8.2 未来优化方向 15

1 系统总体架构

1.1 硬件架构

系统硬件由三大核心模块组成，通过串口和并行总线互联：

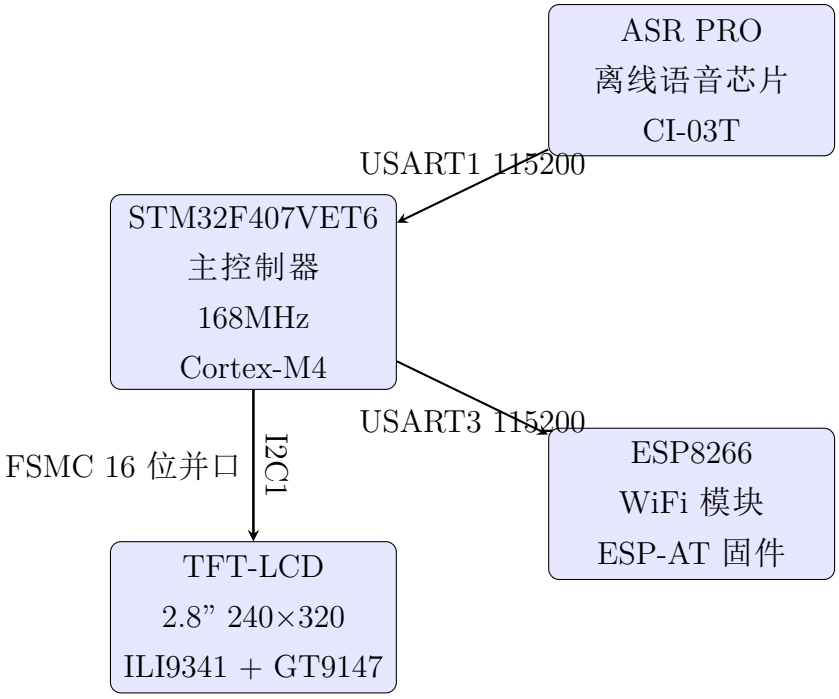


图 1: 系统硬件拓扑

1.2 软件架构

系统基于 FreeRTOS 实现多任务并行调度，共 4 个核心任务：

任务名	功能	优先级	栈大小
defaultTask	系统初始化、外设自检	Normal	4KB
lvglTask	LVGL 图形刷新、UI 事件处理、表情动画、ASR 命令处理	AboveNormal	16KB
esp8266Task	ESP8266 AT 指令交互、WiFi 连接、HTTP/HTTPS 请求	Normal	4KB
asrTask	ASR PRO 串口数据接收与解析	Normal	4KB

表 1: FreeRTOS 任务分配

任务间通过 4 个消息队列通信：

- `asr_queue`：asrTask → lvglTask，传递语音命令 ID
- `esp_ai_req_queue`：lvglTask → esp8266Task，传递 AI 对话请求
- `esp_ai_queue`：esp8266Task → lvglTask，传递 AI 回复文本
- `esp_rx_queue`：esp8266Task → lvglTask，传递系统状态日志

2 用户界面设计

2.1 页面结构

系统包含 6 个 LVGL 页面，通过语音命令或触摸操作切换：

页面	语音命令	功能描述
待机画面 (idle_scr)	“待机画面”	虚拟表情动画（眨眼、眼球游走）
主菜单 (main_scr)	“打开菜单”	4 个功能入口按钮，横向排列为桌面图标入口
系统 (system_scr)	“系统状态” / “设置”	ESP8266 连接状态日志、硬件信息
时钟 (clock_scr)	“时钟”	模拟表盘时钟，包含 12 小时刻度与时分秒指针
日历 (calendar_scr)	“日历”	月视图日历，支持前后翻月与当天高亮
AI 对话 (ai_scr)	“打开对话”	云端 AI 对话界面、拼音输入法

表 2: UI 页面说明

2.2 界面展示

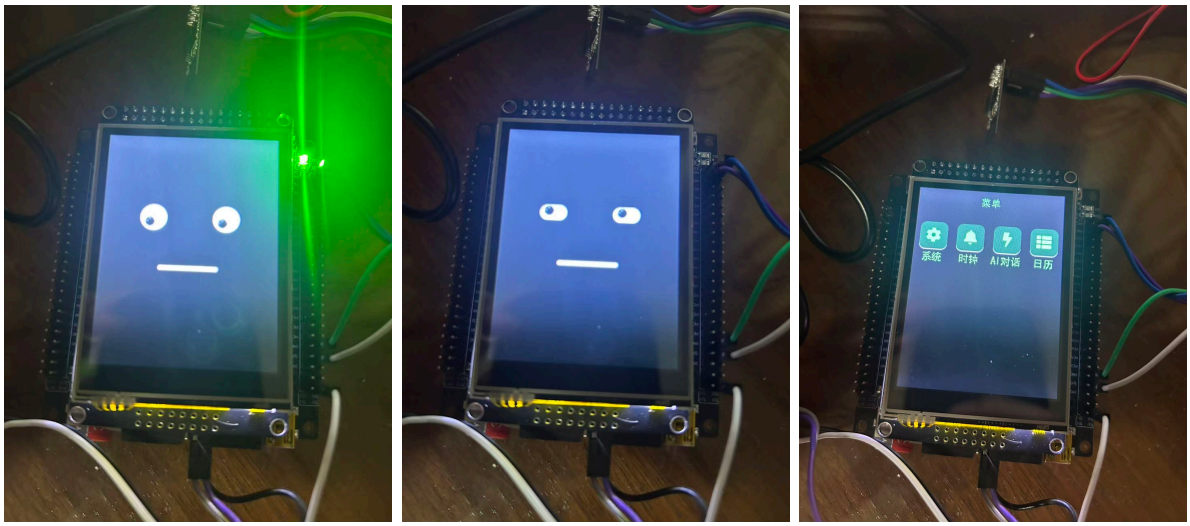


图 2: 待机表情与主菜单界面

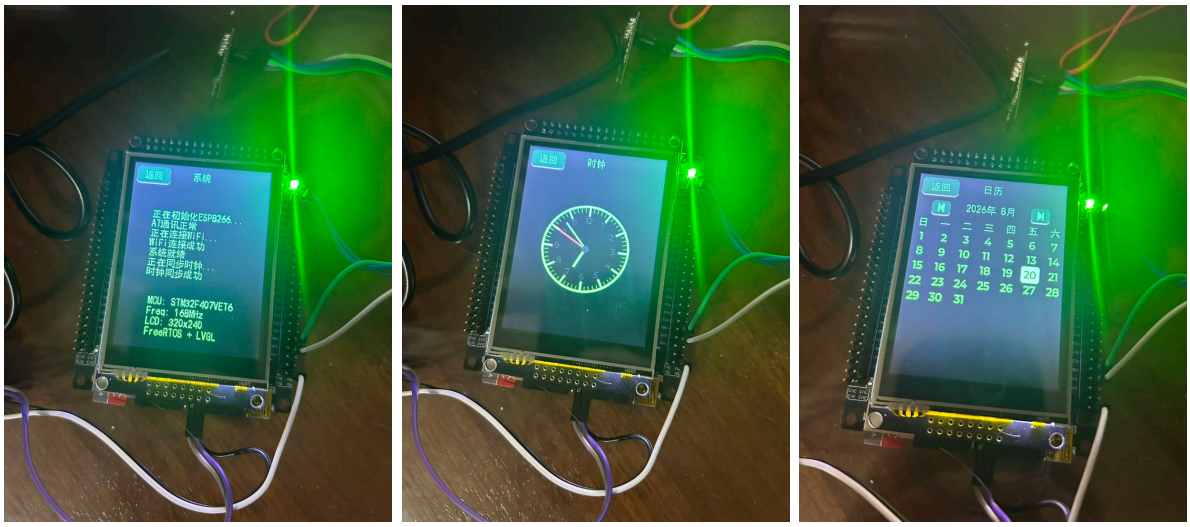


图 3: 系统状态、模拟时钟与日历界面

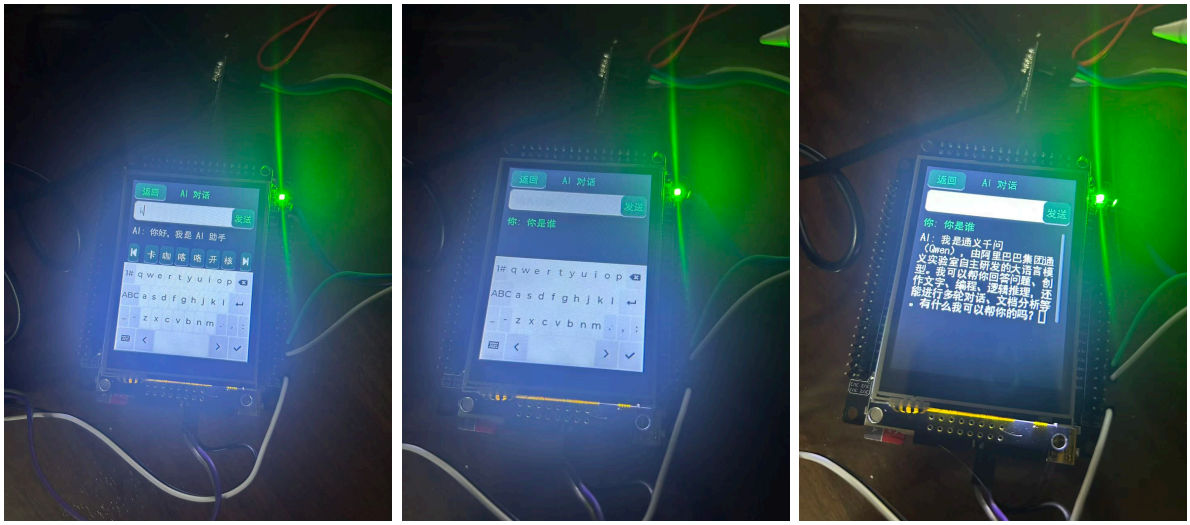


图 4: AI 对话与拼音输入界面

2.3 拼音输入法

在 AI 对话页面实现了完整的拼音输入法，支持 GB2312 一级字库共 3755 个汉字：

- 按拼音音节分组，405 个拼音码表，每组按使用频率排序候选字
- 6 个候选字按钮，实时匹配当前输入拼音串，支持左右翻页
- 支持模糊匹配（输入部分拼音即可显示候选字）
- 虚拟键盘输入，点击候选项自动替换末尾拼音串

2.4 机器人面部表情动画

系统实现了 4 种表情状态的状态机：

状态	触发条件	动画效果
ROBOT_IDLE	默认状态、1.8 秒无事件自动恢复	嘴部细线 (80×5) 闭合，眨眼 (2-5 秒随机)，眼球游走
ROBOT_LISTENING	收到语音命令	嘴部微张 (40×12)，提示正在聆听
ROBOT_THINKING	提交 AI 请求	嘴部圆形 (36×36)，表示思考中
ROBOT_SPEAKING	收到 AI 回复	嘴部交替张合 (60×36/60×8, 180ms 周期)，模拟说话

表 3: 机器人表情状态

表情动画的核心参数：

- 眨眼：2–5 秒随机间隔，闭合 80ms，张开 120ms
- 眼球游走：1.2–2.8 秒随机间隔，移动范围 ±7px，过渡 300ms
- 说话：180ms 周期交替张合
- 自动恢复：无事件 1.8 秒后切回 IDLE

3 语音控制模块

3.1 ASR PRO 离线语音识别

ASR PRO 是天问（TWen51）平台的离线语音识别芯片，基于 CI-03T，支持约 200 条命令词，无需联网即可进行语音识别。

3.1.1 硬件接线

ASR PRO	**STM32F407**	**说明**
TX (PA9)	USART1 RX (PA10)	串口交叉连接
RX (PA10)	USART1 TX (PA9)	串口交叉连接
GND	GND	共地

表 4: ASR PRO 与 STM32 接线

3.1.2 通信协议

ASR PRO 识别到命令词后，通过串口发送 ASCII 十进制数字 + 换行符：

```
// ASR PRO 端代码（上位机 GUI 自动生成）
void ASR_CODE() {
  set_state_enter_wakeup(10000);
  switch (snid) {
   case 1: Serial.write('1'); break;  // 打开菜单
   case 2: Serial.write('2'); break;  // 待机画面
   case 3: Serial.write('3'); break;  // 系统状态
   case 4: Serial.write('4'); break;  // AI对话
   case 5: Serial.write('5'); break;  // 返回
   case 6: Serial.write('6'); break;  // 设置
   case 7: Serial.write('7'); break;  // 时钟
   case 8: Serial.write('8'); break;  // 日历
  }
  Serial.write('\n');
}
```

3.1.3 STM32 端串口接收与解析

STM32 通过 USART1 中断 + DMA 接收 ASR PRO 数据，实现了超时判帧机制：

- 每收到一个字节，记录时间戳
- 收到换行符（0x0A）立即截帧
- 若 20ms 内无新字节且无换行符，自动截帧（兼容无换行符的 ASR 模块）
- 解析时遍历缓冲区，将 ASCII 十进制串转为整数 cmd_id
- 解析完毕调用 `ASR_FlushRx()` 清空缓冲区，重置 HAL 接收指针

4 云端 AI 对话模块

4.1 通信链路

$$\text{STM32F407} \xrightarrow{\text{USART3, 115200}} \text{ESP8266} \xrightarrow{\text{WiFi}} \text{路由器} \xrightarrow{\text{Internet}} \text{阿里云 DashScope}$$

ESP8266 运行 ESP-AT 固件，作为透传网卡，STM32 通过 AT 指令控制其完成 WiFi 连接和 HTTPS 请求。

4.2 AT 指令序列

指令	作用
`AT`	自检，确认模块存活
`ATE0`	关闭回显，避免数据污染
`AT+CWJAP="ssid","pwd"`	连接 WiFi 热点
`AT+CIPSSLSIZE=4096`	调整 SSL 缓冲区大小
`AT+CIPSSLCSNI="host"`	设置 SNI（Server Name Indication）
`AT+CIPSTART="SSL","host",443`	建立 SSL 加密连接
`AT+CIPSEND=<len>`	申请发送 HTTP 请求
··· HTTP 请求体 ···	发送 JSON 格式的 API 请求

表 5: ESP8266 AT 指令序列

4.3 HTTPS 通信的关键坑点

1. **SNI 必须设置**：阿里云 DashScope 走 CDN，同一 IP 对应多个域名。TLS 握手时 ESP8266 必须通过 `AT+CIPSSLCSNI` 告知服务器目标域名，否则服务器无法选择合适的证书，直接关闭连接。

2. **确认 vs 响应**：`AT+CIPSEND` 的 “Recv N bytes” 是模块确认收到数据的字节数，不是远端响应。远端响应以 `+IPD` 开头。

3. **超时设置**：大模型有冷启动延迟，首次请求可能超过 20 秒，接收超时需设为 60 秒。

4. **响应解析**：HTTP 响应包含头部和 JSON body，需从 `"content"` 字段中提取 AI 回复文本，并处理 `\n`、`\"` 等转义字符。

4.4 API 调用

调用阿里云 DashScope 兼容 OpenAI 格式的 Chat Completions API：

```
POST https://dashscope.aliyuncs.com/compatible-mode/v1/chat/
    completions
Authorization: Bearer <API_KEY>
Content-Type: application/json

{
  "model": "qwen3.8-max",
```

```
  "messages": [
    {"role": "user", "content": "你好"},
    {"role": "assistant", "content": "你好！有什么可以帮助你的？"},
    {"role": "user", "content": "今天天气怎么样"}
  ]
}
```

系统维护最多 20 轮对话历史，每次请求将完整历史打包发送，实现多轮对话上下文。

5 关键代码模块

5.1 项目文件结构

```
ai_project/
|-- Core/
|   |-- Inc/                    # 头文件
|   |   |-- FreeRTOSConfig.h    # FreeRTOS 配置
|   |   |-- pinyin_data.h       # 拼音输入法码表 (3755汉字)
|   |   |-- usart.h             # 串口配置
|   |   |-- fsmc.h              # FSMC 并口配置
|   |-- Src/
|       |-- main.c              # 入口，初始化外设
|       |-- freertos.c          # 核心：4个任务 + UI + 表情 + AI对话
|       |-- stm32f4xx_it.c      # 中断服务函数
|       |-- usart.c             # 串口初始化
|-- HARDWARE/
|   |-- ASR_PRO/
|   |   |-- asr_pro.c/h         # ASR PRO 串口驱动
|   |   |-- asrpro.txt          # ASR PRO 端代码参考
|   |-- ESP8266/
|   |   |-- esp8266.c/h         # ESP8266 AT 指令驱动
|   |-- LCD/
|   |   |-- lcd.c/h             # ILI9341 LCD 驱动
|   |-- LVGL/
|   |   |-- lv_port_disp.c/h    # LVGL 显示接口
|   |   |-- lv_port_indev.c/h   # LVGL 触摸输入接口
|   |-- ROBOT_FACE/
|   |   |-- robot_face_anim.c/h # 机器人表情动画
|   |-- TOUCH/
```

```
|       |-- gt9147.c/h          # GT9147 触摸驱动
|       |-- touch.c/h           # 触摸抽象层
|-- Drivers/                    # STM32 HAL 库 + CMSIS
|-- 项目图片/                    # 项目界面截图
|-- DEBUG.md                    # ESP8266 调试笔记
```

5.2 LVGL 显示接口

```c
// lv_port_disp.c - 双缓冲, 240×20 行刷新
static lv_color_t buf1[240 * 20];
static lv_color_t buf2[240 * 20];

static void disp_flush(lv_disp_drv_t *drv, const lv_area_t *area,
                       lv_color_t *color_p) {
    LCD_Color_Fill(area->x1, area->y1, area->x2, area->y2, (u16 *)
       color_p);
    lv_disp_flush_ready(drv);
}
```

5.3 ASR PRO 串口帧解析

```c
// asr_pro.c - 超时判帧 + ASCII 十进制解析
static int ASR_Parse(asr_msg_t *msg) {
    if (!asr.data_ready || asr.rx_len == 0) return 0;

    uint32_t id = 0;
    for (int i = 0; i < asr.rx_len; i++) {
        uint8_t c = asr.rx_buf[i];
        if (c >= '0' && c <= '9')
            id = id * 10 + (c - '0');
        else
            break;
    }

    if (id == 0 || id > 160) { ASR_FlushRx(); return 0; }
    msg->cmd_id = (uint8_t)id;
    ASR_FlushRx();  // 重置缓冲区, 重新开始接收
    return 1;
}
```

5.4 任务间消息队列通信

```
// freertos.c - lvglTask 主循环 (简化)
for (;;) {
    // 1. 读取 ESP8266 状态消息, 更新状态页面
    while (osMessageQueueGet(esp_rx_queueHandle, &m, NULL, 0) == osOK
       ) { ... }

    // 2. 读取 AI 回复, 追加到对话历史
    if (osMessageQueueGet(esp_ai_queueHandle, ai_buf, NULL, 0) ==
       osOK) {
        chat_add_message("assistant", ai_buf);
        robot_face_set_state(ROBOT_SPEAKING);  // 触发说话表情
    }

    // 3. 读取 ASR 语音命令, 切换页面
    if (osMessageQueueGet(asr_queueHandle, &asr_msg, NULL, 0) == osOK
       ) {
        robot_face_set_state(ROBOT_LISTENING);
        switch (asr_msg.cmd_id) {
            case ASR_CMD_IDLE:  lv_scr_load(idle_scr); break;
            case ASR_CMD_MAIN_MENU: lv_scr_load(main_scr); break;
            // ... 其他页面切换
        }
    }

    // 4. 表情自动恢复: 1.8秒无事件 → IDLE
    robot_face_tick();
    lv_timer_handler();
    osDelay(5);
}
```

6 调试与问题解决

6.1 ASR PRO 串口通信问题

问题：ASR PRO 语音命令无法切换 STM32 屏幕页面。

根因：ASR_Parse 解析完一帧数据后仅将 `data_ready` 置 0，未重置 `rx_len` 和 HAL 接收指针。导致：

- `rx_len` 持续累积增长
- `HAL_UART_Receive_IT` 指针越跑越偏，新数据存不到 `buf[0]`
- 解析器始终读取 `buf[0]` 的旧数据，同一命令被无限重复解析

解决：在 `ASR_Parse` 末尾调用 `ASR_FlushRx()`，一次性重置所有状态：`rx_len=0`、清空缓冲区、`HAL_UART_Receive_IT` 重新指向 `buf[0]`。

6.2 ESP8266 HTTPS 连接问题

详见 `DEBUG.md`，核心问题与解决方案：

- **SNI 缺失：**ESP8266 老版本固件不支持 SNI，导致 TLS 握手失败，服务器直接关闭连接。通过在 `AT+CIPSTART` 前调用 `AT+CIPSSLCSNI` 解决。
- **确认误判：**将 `AT+CIPSEND` 的确认信息误判为远端响应，导致解析逻辑错误。
- **冷启动超时：**大模型首次调用需加载模型，超时从 20 秒调整为 60 秒。

7 技术指标

指标	参数
CPU 主频	168MHz, Cortex-M4
RAM 使用	约 80KB / 192KB (含 LVGL 双缓冲 9.6KB)
Flash 使用	约 400KB / 512KB (含 3755 汉字拼音码表)
LCD 刷新率	约 30–50 FPS (LVGL 双缓冲)
ASR 识别延迟	$<$ 500ms (离线)
AI 对话延迟	3–20 秒 (取决于网络和模型负载)
串口波特率	115200 (USART1 接 ASR PRO, USART3 接 ESP8266)
WiFi 标准	802.11 b/g/n, 2.4GHz

表 6: 系统技术指标

8 总结与展望

8.1 项目成果

本项目成功实现了一款集语音控制、虚拟表情动画、模拟时钟、日历与云端 AI 对话于一体的智能交互终端。主要成果包括：

1. 基于 FreeRTOS 的多任务系统架构，任务间通过消息队列解耦，模块化程度高
2. 基于 LVGL 的图形界面，包含 6 个页面，支持拼音输入法（3755 汉字）
3. 机器人面部表情动画系统，4 种状态自动切换，具有情感化表达
4. ASR PRO 离线语音命令控制，支持 8 条页面切换命令
5. ESP8266 通过 HTTPS 连接阿里云 DashScope，实现多轮 AI 对话

8.2 未来优化方向

1. **语音输入升级：**将 ASR PRO 扩展为全双工语音对话，或替换为 ESP32-S3 实现云端流式语音识别
2. **表情丰富化：**增加更多表情状态（开心、难过、惊讶、困惑），添加眉毛/腮红等面部元素
3. **传感器扩展：**集成温湿度、光照、人体感应等传感器，实现环境感知
4. **物联网功能：**通过 WiFi 获取天气、新闻、时间等信息，扩展实用功能
5. **OTA 升级：**实现通过 WiFi 的固件远程升级